

1 Markovian Drone

1.1 Optimal Value Heatmap Plot

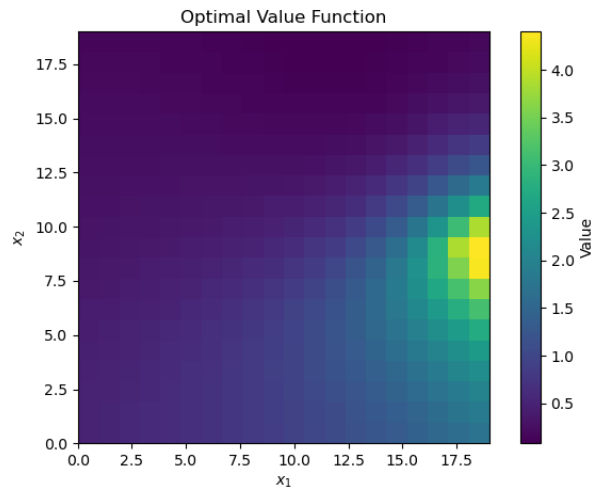


Figure 1: Optimal Value Heatmap Plot

1.2 Optimal Policy Heatmap Plot

As seen in Fig. 2, the simulated drone is following actions depending on the ego-state, denoted by the heatmap. Sometimes the policy is overridden by the storm, that is the reason why the drone is taking sometimes different actions than those of the policy. Also, the simulation runs for 100 steps, so even after reaching the goal, the drone may circle around it.

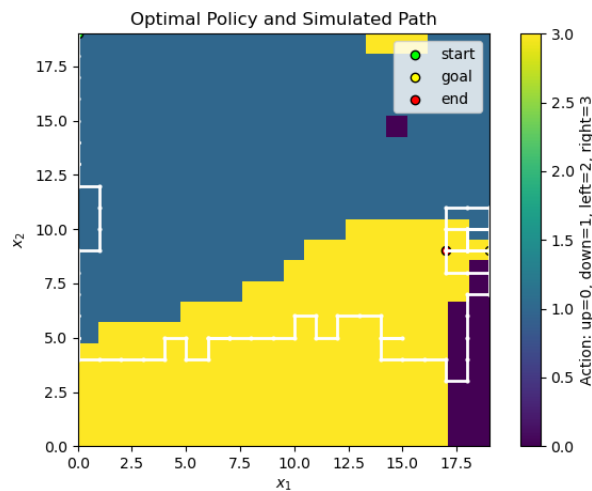


Figure 2: Optimal Policy Heatmap Plot

1.3 Code

Code in Appendix.

Code Appendix

Homework part	Code file
2.1	markovian_drone.py

markovian_drone.py

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 ACTIONS = np.array([ [0,1], # up
5                      [0,-1], # down
6                      [-1,0], # left
7                      [1,0], # right
8                      ])
9
10 def idx_to_state(s,n=20):
11     x1 = s % n
12     x2 = s // n
13     return np.array([x1,x2])
14
15 def move(x,a,n=20):
16     x_next = x+a
17     x_next = np.clip(x_next,0,n-1)
18
19     return x_next[1]*n + x_next[0]
20
21 def value_iteration(n,eps=1e-6):
22     sigma = 10
23     gamma = 0.95
24     x_eye = np.array([15,15])
25     x_goal = np.array([19,9])
26     num_states = n*n
27     V = np.zeros(n*n)
28     opt_policy = np.zeros(n*n, dtype=int)
29
30     T = np.zeros((n*n,4,n*n)) # current, action, next
31     R = np.zeros(num_states)
32     R[x_goal[1] * n + x_goal[0]] = 1
33
34     for s in range(num_states):
35         # storm_influence
36         x = idx_to_state(s)
37         omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))
38
39         # Build transition
40         for a in range(4):
41             # specified action
42             next_s = move(x,ACTIONS[a])
43             T[s,a, next_s] += 1 - omega
44
45             # random action
46             for a_rand in range(4):
47                 next_s_rand = move(x,ACTIONS[a_rand])
48                 T[s,a,next_s_rand] += omega/4
49
50     diff = np.inf
51     while diff > eps:
52         V_new = np.zeros_like(V)
```

```

54     for s in range(num_states):
55         q = np.zeros(4)
56
57         for a in range(4):
58             q[a] = np.sum(T[s,a,:] * (R + gamma*V))
59
60         opt_policy[s] = np.argmax(q)
61         V_new[s] = q[opt_policy[s]]#np.max(q)
62
63     diff = np.max(np.abs(V_new - V))
64     V = V_new
65
66     return V, opt_policy
67
68 def simulate_MDP(policy,n):
69     sigma = 10
70     x_eye = np.array([15,15])
71     x_init = np.array([0,19])
72     time_steps = 100
73     path = [x_init.copy()]
74
75     x = x_init.copy()
76     for _ in range(time_steps):
77         s = x[1] * n + x[0]
78         action = policy[s]
79         omega = np.exp(-np.linalg.norm(x - x_eye)**2 / (2 * sigma**2))
80
81         if np.random.rand() < omega:
82             action = np.random.randint(4)
83
84         next_s = move(x,ACTIONS[action],n)
85         x = idx_to_state(next_s,n)
86         path.append(x.copy())
87
88     return np.array(path)
89
90 def plot_heatmaps(V,policy,path,n):
91     V_grid = V.reshape((n,n))
92     policy_grid = policy.reshape((n,n))
93
94     plt.figure()
95     plt.imshow(V_grid, origin='lower', extent=[0,n-1,0,n-1])
96     plt.colorbar(label='Value')
97     plt.xlabel('$x_1$')
98     plt.ylabel('$x_2$')
99     plt.title('Optimal Value Function')
100    plt.tight_layout()
101    # plt.show()
102    plt.savefig("optimal_value_heatmap.png")
103
104    plt.figure()
105    plt.imshow(policy_grid, origin='lower', extent=[0,n-1,0,n-1], vmin=0, vmax=3)
106    plt.colorbar(label='Action: up=0, down=1, left=2, right=3')
107    plt.plot(path[:,0], path[:,1], color='white', linewidth=2, marker='o', markersize=2)
108    plt.scatter(path[0,0], path[0,1], color='lime', edgecolor='black', label='start')
109    plt.scatter(19,9, color='yellow', edgecolor='black', label='goal')
110    plt.scatter(path[-1,0], path[-1,1], color='red', edgecolor='black', label='end')
111    plt.xlabel('$x_1$')
112    plt.ylabel('$x_2$')
113    plt.title('Optimal Policy and Simulated Path')
114    plt.legend()
115    plt.tight_layout()
116    plt.savefig("policy_heatmap_path.png")

```

```
117
118 def main():
119     n = 20
120
121     # Part A
122     V,policy = value_iteration(n)
123
124     # Part B
125     path = simulate_MDP(policy,n)
126
127     plot_heatmaps(V,policy,path,n)
128
129
130 if __name__ == '__main__':
131     main()
```